

Chapter 7: Dimensionality Reduction & Latent Spaces

One concept, one exercise per slide — practice as you go

Concept: What PCA Is Trying to Do

- Find the direction in feature space along which the data has MAXIMUM variance
- That direction is the first principal component
- Subsequent components are the next-best directions, each ORTHOGONAL to all previous ones
- Projecting onto the top-k components compresses D features down to k with minimal information loss



Exercise: A dataset has 1000 features, but the first 2 principal components explain 95% of the total variance. What does this suggest about the data's true underlying dimensionality?

Concept: The Covariance Matrix and Its Eigenvectors

- Center the data first (subtract the mean from every feature)
- The covariance matrix $C = (1/(N-1)) \tilde{X}^T \tilde{X}$ captures how features vary together
- Principal components are exactly the EIGENVECTORS of C
- Each eigenvector's eigenvalue equals the variance captured along that direction

 **Exercise: Why must the data be mean-centered before computing the covariance matrix for PCA?**

Concept: Power Iteration — Finding the Dominant Eigenvector

- Repeatedly apply $v \leftarrow Cv / ||Cv||$ — this converges to the eigenvector with the LARGEST eigenvalue
- Convergence rate depends on $|\lambda_2/\lambda_1|$ — well-separated top eigenvalues converge fast
- After finding one component, DEFLATE it out to find the next: $C' = C - \lambda_1 v_1 v_1^T$



Exercise: A covariance matrix has top two eigenvalues $\lambda_1=10$ and $\lambda_2=9.9$ (very close together). Would you expect power iteration to converge quickly or slowly to the first principal component, and why?

Concept: The SVD-Covariance Duality

- Economy SVD: $\tilde{X} = U \cdot \Sigma \cdot V^T$ (mean-centered data)
- The columns of V are EXACTLY the eigenvectors of the covariance matrix — no eigensolver needed
- The scores (projections) satisfy $Z = \tilde{X}V = U\Sigma$ — read directly off the SVD factors
- This is why production PCA implementations use SVD rather than explicitly forming the covariance matrix



Exercise: Forming the $D \times D$ covariance matrix costs $O(ND^2)$ and can be numerically unstable for large D . Why does computing the SVD directly on the $N \times D$ data matrix avoid this problem?

Concept: The Eckart-Young Theorem — Optimal Low-Rank Approximation

- Truncating the SVD to the top k singular values gives the BEST possible rank- k approximation
- "Best" means minimal reconstruction error, in both Frobenius and spectral norm
- Fraction of variance explained by the top k components: $\sum_{i=1}^k \sigma_i^2 / \sum_j \sigma_j^2$



Exercise: A matrix has singular values $\sigma_1=5$, $\sigma_2=3$, $\sigma_3=1$. What fraction of the squared Frobenius norm does the rank-1 approximation (using only σ_1) capture?

Concept: Condition Number and Numerical Stability

- Forming $\tilde{X}^T \tilde{X}$ SQUARES the condition number: $\kappa(C) = \kappa(\tilde{X})^2$
- A moderately ill-conditioned $\tilde{X}$ can become severely ill-conditioned once squared
- This costs real floating-point precision — computing the SVD directly avoids the squaring



Exercise: A data matrix has condition number $\kappa(X)=1000$. What is the condition number of its covariance matrix, and roughly how many decimal digits of precision are lost by forming it explicitly (in float64, machine epsilon $\approx 1e-16$)?

Concept: Randomized SVD for Large-Scale Data

- Exact SVD costs $O(N \cdot D^2)$ — impractical for very large D
- Randomized SVD projects onto a random low-dimensional sketch first, then computes a small SVD
- Cost drops to about $O(N \cdot D \cdot k)$ for the top k components — dramatically cheaper when $k \ll D$



Exercise: You need the top 50 components of a matrix with $D=10,000$ features and $N=1,000$ rows. Roughly how much cheaper is randomized SVD ($O(N \cdot D \cdot k)$) than the exact SVD ($O(N \cdot D^2)$) here?

Concept: Mistake — Forgetting to Center the Data

- Running PCA on raw, uncentered data is a common and completely silent bug
- The resulting "components" are contaminated by the data's distance from the origin
- Always subtract the per-feature mean before computing covariance or running SVD for PCA



Exercise: A colleague runs PCA on raw sensor readings that all happen to be large positive numbers (e.g., 1000-1010) and gets a strange first component pointing almost exactly along the direction of the sensors' average reading. What did they forget to do?

Mastery Check: Are You Ready to Move On?

If you can answer every question below confidently, you have mastered Chapter 7.

1. Explain what a principal component is and why each one must be orthogonal to the previous ones.
2. Explain why PCA requires mean-centering the data first.
3. Explain why the principal components are the eigenvectors of the covariance matrix.
4. Explain how power iteration finds the dominant eigenvector and why the eigenvalue ratio controls its speed.
5. State the SVD-covariance duality and explain why production PCA uses SVD instead of an explicit covariance matrix.
6. State the Eckart-Young theorem and compute a rank-k approximation's captured variance fraction by hand.
7. Explain why forming $X^T X$ squares the condition number, and why this matters numerically.
8. Explain when and why randomized SVD is preferred over the exact SVD.
9. Diagnose a PCA result that looks wrong because the data was never mean-centered.
10. Implement PCA from scratch via power iteration and verify it against a library's SVD-based PCA.